

EPYCfail Detailed Microarchitecture Specification

Tanmay Garudadri
tanmayg3@illinois.edu

Zaheen Jamil
zaheenj2@illinois.edu

Reet Sinha
reet2@illinois.edu

Introduction— Our core, EPYCfail, implements a superscalar, out-of-order (OoO) RV32IM processor. That is, our core exploits instruction-level parallelism by potentially executing instructions out-of-order as well as fetching multiple instructions per cycle. This paper will provide an overview of our core, then detail how the core progressed across 4 design checkpoints. Then, we will discuss advanced features our core supported, such as our split load-store queue (LSQ), our banked branch prediction pattern history tables, our instruction cache prefetcher, and more. Finally, we will discuss directions for future optimizations. Implementing explicit-register-renaming (ERR), including the reorder buffer, issue queue, and rename logic, followed by going superscalar and supporting advanced features gave us a glimpse into how industry high-performance cores might work, and the tradeoffs that designers must consider.

I. PROJECT OVERVIEW

From the start, we split up the work. Tanmay worked on the rename, dispatch, and issue logic, and execute optimizations. Reet worked on the rest of the frontend and the ROB. Zaheen worked on anything relating memory instructions (i.e. split load-store queue, data cache optimizations). We built our core with advanced features in mind from the very beginning, as seen with the split LSQ, but also with branch prediction as well. The result was that we had completed CP3 functionality even before the CP2 deadline, at least simulation-wise. Then, while we made the jump to superscalar, we also started synthesizing our core and spent a week optimizing critical paths, mostly in fetch and the split LSQ. Staying under the 300k area limit and pushing our frequency dominated our design decisions in the last 2 weeks, even preventing us from including implemented features.

II. DESIGN DESCRIPTION

A. Overview

At a high level, EPYCfail is organized as a 4-wide fetch frontend that feeds a 2-wide rename/dispatch middle end, which in turn feeds a distributed issue/execute backend with six functional units, three CDBs, and a 2-wide in-order commit ROB. The width mismatch between the frontend and the rest of the core is deliberate: it lets the frontend capture instruction-cache bandwidth in larger bursts while the backend consumes instructions at a steady, sustainable rate. The fetch queue serves as the decoupling buffer between these two domains, smoothing out frontend bursts and hiding short backend stalls.

Our core begins with a 4 way superscalar fetch, with the icache returning up to 4 words for every upward facing request. The instruction cache is a direct-mapped cache with 32-byte (eight-word) lines, optimized for the fetch access pattern by returning an entire line per request and allowing the fetch stage to slice any aligned 1-to-4-word group from it. To prevent starvation and fill as many words as possible, a consistent problem in our design, we made a few optimizations. We designed a stream prefetcher that fetches up to $N+2$ lines, implemented a 2 entry victim cache, incorporated non-blocking prefetch capability with MSHRs. Together, these mechanisms target three distinct sources of frontend bubble: cold or capacity misses on sequential code (addressed by the next-line and degree-2 prefetcher), conflict misses in the direct-mapped I-cache (addressed by the victim cache), and the inability to overlap demand fetches with outstanding misses (addressed by the prefetch MSHRs).

Fetch instructions feed into our fetch queue (fetchq) which can enqueue up to 4 instructions in a cycle and dequeue up to 2. Each fetch queue entry holds a full 4-wide fetch group along with at most one branch-prediction metadata record for the group, since the frontend predicts at most one control-flow instruction per packet. Although the theoretical maximum rates of enqueueing and dequeueing may result in the fetchq filling up indefinitely, performance analysis showed us that the backend of the processor was consistently faster at executing instructions than the fetch stages were at filling the fetchq. In other words, the fetchq tended toward empty rather than full in the steady state, which validated the choice of a wider frontend over a wider backend. Therefore, such a setup was optimal.

Following the fetchq, instructions followed the typical explicit register renaming architecture flow. Decode produces up to two `ctrl_words` per cycle, which rename then converts into a dispatch packet by looking up source physical registers in the RAT, allocating physical destination registers from the free list, and resolving any intra-packet dependencies (a slot-1 instruction reading a register written by slot 0, or both slots writing the same architectural register) directly in the rename priority network. The renamed packet is captured in a one-entry dispatch staging register, which serves as a timing-isolation boundary between the rename logic and the issue queues. We incorporated a 2 way superscalar middle end that connects to our backend. With 6 functional units (including LSU), 5 issue queues and 5 CDBs, the processor is able to execute and complete many instructions in parallel. The issue queues are FU-typed rather than unified: there are dedicated queues for ALU0, ALU1, branch, M-extension (shared by MUL and DIV), and memory. This avoids the area and timing cost of a single large CAM and allows each queue to be

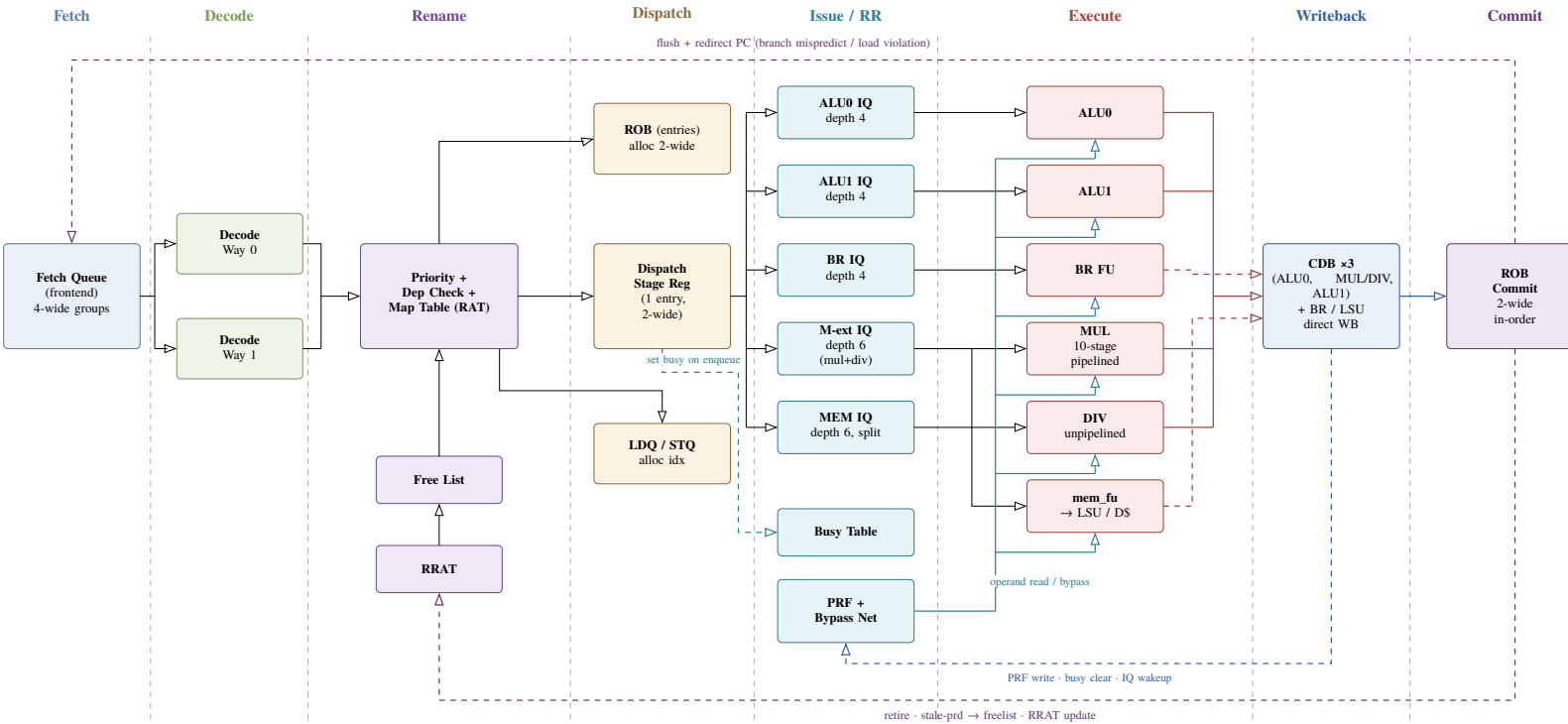


Fig. 1. Stage overview of the OoO backend from fetch queue through commit. Stages are separated by dashed vertical guides representing pipeline registers. Solid black arrows show forward instruction/dataflow. Teal arrows show PRF/bypass operand supply into the functional units. Red solid arrows show CDB-producing writebacks; red dashed arrows show BR/LSU direct writeback paths. Blue dashed arrows show CDB feedback for PRF writes, busy-table clears, and IQ wakeup. Purple dashed arrows show commit-time recovery and retirement updates.

sized and parameterized independently based on its expected occupancy and the latency characteristics of its consumer.

Our re-order buffer had the ability to dequeue 2 completed instructions every cycle, with lane-1 commit gated on lane-0's outcome to handle mispredicts and load-violation replays. This gating is important for correctness: if the lane-0 instruction is a mispredicted branch or a load that violated memory ordering, the lane-1 instruction must not retire, since a flush is about to redirect the pipeline and any architectural state changes from lane 1 would corrupt the recovered state. In practice, this gating costs very little IPC because mispredicts and load violations are rare, and dual retire on the common case of two ordinary instructions still gives most of the benefit of a 2-wide commit width.

B. Milestones

a) Checkpoint 1:

The requirements for checkpoint 1 included writing a parametrizable queue, making a diagram of our planned processor, implementing a cacheline adapter to convert DRAM bursts to 256 bit cache lines, and to implement a single fetch stage.

We successfully completed all of these requirements within the deadline, working on most of the code and the diagram over spring break to ensure it would be complete early.

First we made our diagram and made sure to plan out everything well. Zaheen worked through all the LSU logic that we might need such that we could implement a full split-LSQ, with non-blocking capability as well as all the other plans for LSU advanced features. Tanmay worked through planning

for rename and dispatch logic as well as issue queue and the physical register file. He then implemented the parametrizable queue. Reet planned the execution units as well as the fetch stage which he promptly implemented to meet the CP1 deadline.

b) *Checkpoint 2:* For this deadline we began writing our first ever draft of a single scalar out of order processor. We first ported our decode stage from prior implemented the rename and dispatch logic. In parallel we designed efficient CAM logic with only 2 bit lookups (valid and ready) for the issue queues to minimize the critical path. We then tested just the issue and rename logic and found that it could run at around 800 MHz by itself. We were satisfied with this critical path.

Then we wrote a basic ALU and used DesignWare IPs for the other functional units (multiply and divide). In addition we added early wakeup signals to the ALU in order to get 1 IPC on subsequent ALU instructions.

At the time we were also trying to minimize the number of CDBs that we had to minimize the ROB area as we hadn't seriously begun calculating an area budget yet. Therefore we combined the multiply and divide CDBs using an arbiter but gave the ALU and BR instructions their own dedicated CDBs.

As this checkpoint didn't yet require memory instructions, we only created an identical issue queue for the mem_fu and made no further preparations.

c) *Checkpoint 3:* For this checkpoint we implemented speculative execution on control instructions, as well as a basic LSU that handled memory instructions at commit.

Advanced Feature Name	Performance Before Implementation	Performance After Implementation	Kept / Deleted
Split LSQ	0.4 IPC Coremark	0.5 IPC Coremark	Kept
Pipelined dcache	0.5 IPC Coremark	0.51 IPC Coremark	Kept
Post-Commit Store buffer	0.51 IPC Coremark	0.514 IPC Coremark	Kept
Pre-Commit Store forwarding + Write Coalescing in MSHRs	0.514 IPC Coremark	0.514 IPC Coremark	Kept
Non-Blocking, pipelined dcache	0.514 IPC Coremark	0.525 IPC Coremark	Kept
Next-Line Prefetcher	0.68 IPC Coremark	0.72 IPC Coremark	Kept
Pipelined icache	N/A (we started with pipelined icache)	N/A	Kept
G-Share	0.32 IPC Coremark	0.39 IPC Coremark	Kept
Superscalar (4 way fetch, 2 way otherwise)	0.72 IPC Coremark, 580 MHz	0.77 IPC Coremark, 450 MHz	Kept
Return Address Stack	0.39 IPC Coremark	0.41 IPC Coremark	Kept
Banked D-Cache	0.8 Coremark IPC, 615 MHz	0.8 Coremark IPC, 590 MHz	Deleted

For control instructions, we implemented a basic gshare branch predictor with the smallest SRAM configuration possible.

We tried to improve performance by expanding the history register to 64 bits. On paper, more history seemed better but when we later changed it back to 12 bits we noticed significant performance gains. As it turned out, the extra history was having a polluting effect.

For JALR, we implemented a return address stack and immediately noticed big performance gains. Later we added auipc/jalr fusion and noticed even more gains.

For misprediction correction, we added a retired register alias table (RRAT) that would be updated on commit. Whenever a mispredicted instruction reached the head of the ROB, we copied the RRAT into the RAT.

For many reasons this was not the ideal implementation for our LSU and we wanted to change it as soon as possible. Handling memory at commit required us to stall commits at the head of the ROB until the load/store completed which might take many cycles. We were also forced to execute them in-order and also failed to exploit the time between their issue/dispatch up to them reaching the head of the ROB. As expected, performance was quite subpar.

But with these additions our processor could execute all instructions including memory and control (albeit not well). We had finished CP3 by the CP2 deadline and so had extra time to implement our advanced features.

III. ADVANCED DESIGN FEATURES

A. Table of Advanced Features and Performance Implications

B. Split LSQ + Non-blocking, pipelined data cache

a) Design:

The final design of our split LSQ and non-blocking data cache is detailed in Fig. 2. It is speculative, in that it executes loads before all older store addresses are resolved. Each queue is age-ordered, due to being allocated at dispatch. This allows loads which are older to be prioritized. The store queue (STQ) has an integrated post-commit store buffer (PCSB). This is implemented by storing an additional pointer for the STQ separating pre-commit and post-commit stores. This allows the ROB to continue committing past stores that haven't been able to fire to the data cache (of course, eventually the STQ will fill and rename will be stalled if the stores never drain). Loads go through a 2-stage pipeline: load candidate selection

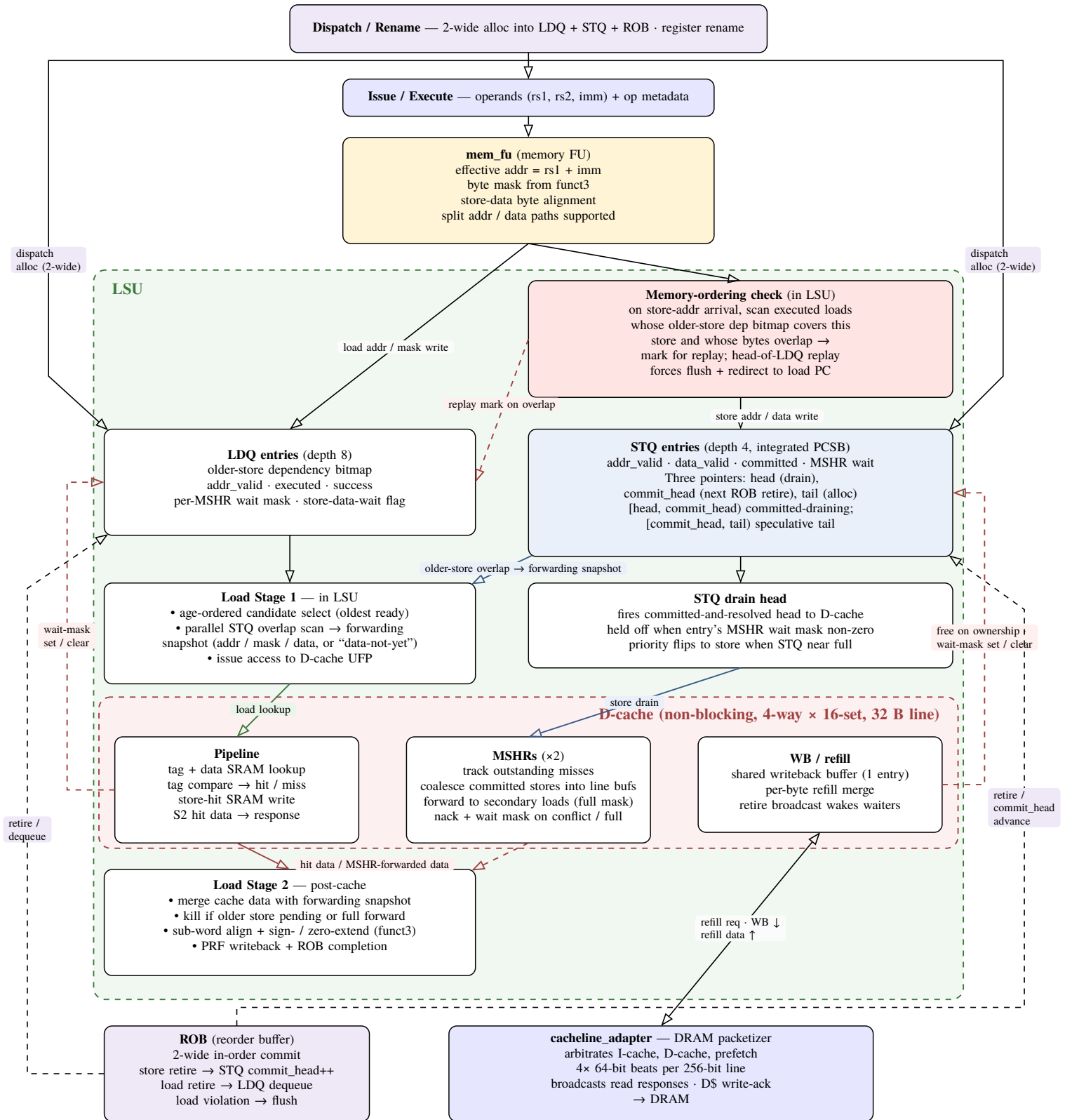


Fig. 2. LSU and D-cache interaction. Loads traverse a two-stage pipeline: Stage 1 (in the LSU) selects the oldest ready load, scans the STQ in parallel for older-store overlap to build a forwarding snapshot, and issues the access; Stage 2 (post-cache) merges cache data with the snapshot, kills the result if an older store’s data is still pending or the forward fully covers the load, sub-word-aligns and extends, then writes back the PRF and signals ROB completion. The STQ retains committed stores until their D-cache writes complete (integrated PCSB); its three pointers separate the committed-draining region from the speculative tail. On a store-address arrival the LSU runs a memory-ordering check against the LDQ — a head-of-LDQ replay forces flush and redirect. The non-blocking D-cache supports MSHR-side store-to-load forwarding (full mask coverage), per-byte refill merge, and a shared writeback buffer; the cacheline_adapter packetizes 256-bit lines into four 64-bit DRAM beats. Edge legend — solid edges are forward data / request paths (green: load issue; blue: store issue and STQ→LDQ forwarding; black: generic flow including Dispatch alloc into LDQ/STQ). Dashed red edges are D-cache back-channels: MSHR wait-mask set/clear, free-on-ownership, MSHR-forwarded load data, and the memory-ordering replay mark. Dashed black edges are ROB commit signals: load retire / dequeue into the LDQ, and store retire that advances the STQ commit_head. Dashed cluster boundaries delimit the LSU and D-cache groupings; they do not represent signals.

and data formation. The first stage selects the oldest load which has not already been sent to the data-cache nor needs to be forwarded later from a store that had an address match. The second stage receives data from the data cache, and merges that with potential forwarded data from the STQ as well as masking the data based on `funct3`. The data cache is non-blocking and supports a parametrizable number of Miss Status and Handling Registers (MSHRs). It also has a kill signal that prevents MSHRs from being allocated on load misses upon identifying that the data should be forwarded from the STQ in the 2nd stage. It also forwards from MSHRs to matching loads, which is correct because stores sent to the data cache must have already been committed, and therefore older than any requesting load. There is also support for allocating 2 memory instructions at once and committing 2 loads at once. Most design tradeoffs were due to timing and area constraints. The load-pipe is 2 stages due to timing, and the LDQ and STQ sizes are 8 and 4 respectively due to timing and area. Because our baseline split LSQ was speculative with load-forwarding, we could not compare to a baseline split LSQ which was not speculative. The simpler, non-speculative split LSQ would likely allow for a 1-stage load-pipe, reducing the load-use latency by 1 cycle but possibly stalling loads for much longer.

b) *Performance Analysis:*

Our initial split LSQ was implemented relatively early on with a 1-stage load pipe. At this point, our core only had about 0.4 IPC on coremark. Replacing the unified LSQ increased IPC by 0.1 up to 0.5 on Coremark. The non-blocking data cache was implemented soon after, and saw smaller but not negligible gains of between 0.01 and 0.02 IPC. At this point, we did not have detailed performance counters, but this was likely due to either the ROB window or LDQ being too small to be able to overlap many loads. Later, when reducing critical paths to push our core's frequency, we were forced to split the load process into 2 stages, which caused 0.02 - 0.04 IPC decreases on most benchmarks. However, this allowed our core to reach 600 MHz so the frequency gain outweighed the IPC loss. At the core's current state, unresolved loads are still the biggest blocker of ROB commits. While for some programs, this is due to frontend stalls, for AES-SHA this was due to contention on the data cache port, due a high density of loads and stores. In FFT, MSHR perf trackers reveal that loads are frequently replaying due to MSHRs being full. We experimented with banked data caches but only saw small improvements. In less than 5% of cycles were 2 loads able to issue to the 2 separate banks simultaneously. Overall, more MSHRs, larger STQ size, and larger ROB size would benefit the latency of memory operations the most but these bring significant area and timing costs.

C. 4-Way Superscalar, 2-Stage Frontend

a) *Motivation:* By the end of Checkpoint 3, we observed that our fetch queue was frequently empty when using a simple scalar frontend, indicating that the frontend was limiting instruction supply. We first attempted to increase bandwidth with a single-stage, 2-way superscalar fetch unit, but this quickly became the critical timing path. To address this, we

redesigned fetch as a two-stage pipeline with redirects for branches, `jal`, and `jalr` resolved in the second stage. However, the added late-redirect penalty, combined with the lack of support for misaligned fetches across cache-line boundaries, limited the performance gains from the wider frontend. To better utilize frontend bandwidth and amortize these penalties, we expanded both fetch and the fetch queue to support 4-wide operation.

b) *Design:*

The final frontend is a 4-wide, two-stage fetch pipeline feeding a 2-wide decode / rename backend. Fetch produces up to four instructions per cycle, while the backend consumes up to two. Each fetch queue entry stores one 4-wide fetch group, per-slot valid bits, and at most one branch-prediction metadata record.

A PC register tracks the next fetch address. A request issues when the fetch queue has space, the frontend pipeline can accept a packet, the branch predictor is ready, and the I-cache is ready. The pipelined I-cache returns a 256-bit cache line containing eight 32-bit instructions. Fetch slices up to four instructions from the returned line using the response PC, since a newer request may already be in flight when an older response returns.

The frontend does not fetch across cache-line boundaries. If the PC starts near the end of a 32-byte line, fewer than four instructions are valid. For example, word offset 5 gives three valid instructions, while word offset 7 gives one. This simplifies timing and slicing, but reduces effective fetch bandwidth near line boundaries.

The first frontend stage performs lightweight predecode over the four slots. It identifies conditional branches, `jal`, and `jalr`, computes immediates, and selects the first control-flow instruction. Instructions after that control-flow instruction are masked off. This limits the frontend to one predicted control-flow instruction per fetch packet.

Conditional branches use a gshare predictor with a speculative global history register and banked PHTs (4 banks, or one per slot) of 2-bit saturating counters. The PHT bank is selected from branch slot alignment, and the PHT index is computed by XORing PC bits with global history. The packet stores the PHT index, bank, counter value, branch slot, and GHR snapshot for commit-time training.

The initial gshare design used 1024 entries per bank with 1R1W SRAM ports. This simplified simultaneous fetch lookup and commit training, but cost about 51k area. We reduced the predictor to 256 entries per bank and changed the PHTs to 1RW ports with a small training FIFO / write queue. Area dropped to about 13k, while CoreMark branch accuracy only fell from 93.1% to 92.9%.

The predictor updates the speculative GHR during fetch. At commit, the actual branch outcome updates the PHT counter. On a misprediction, the saved GHR snapshot restores the speculative history. The training FIFO keeps commit-side predictor writes off the fetch critical path.

Unconditional `jal` instructions are handled by computing `pc + imm` in the frontend. For `jalr`, the frontend supports `auipc/jalr` fusion and return prediction. Fusion computes the target early when a nearby `auipc` feeds the `jalr`. The

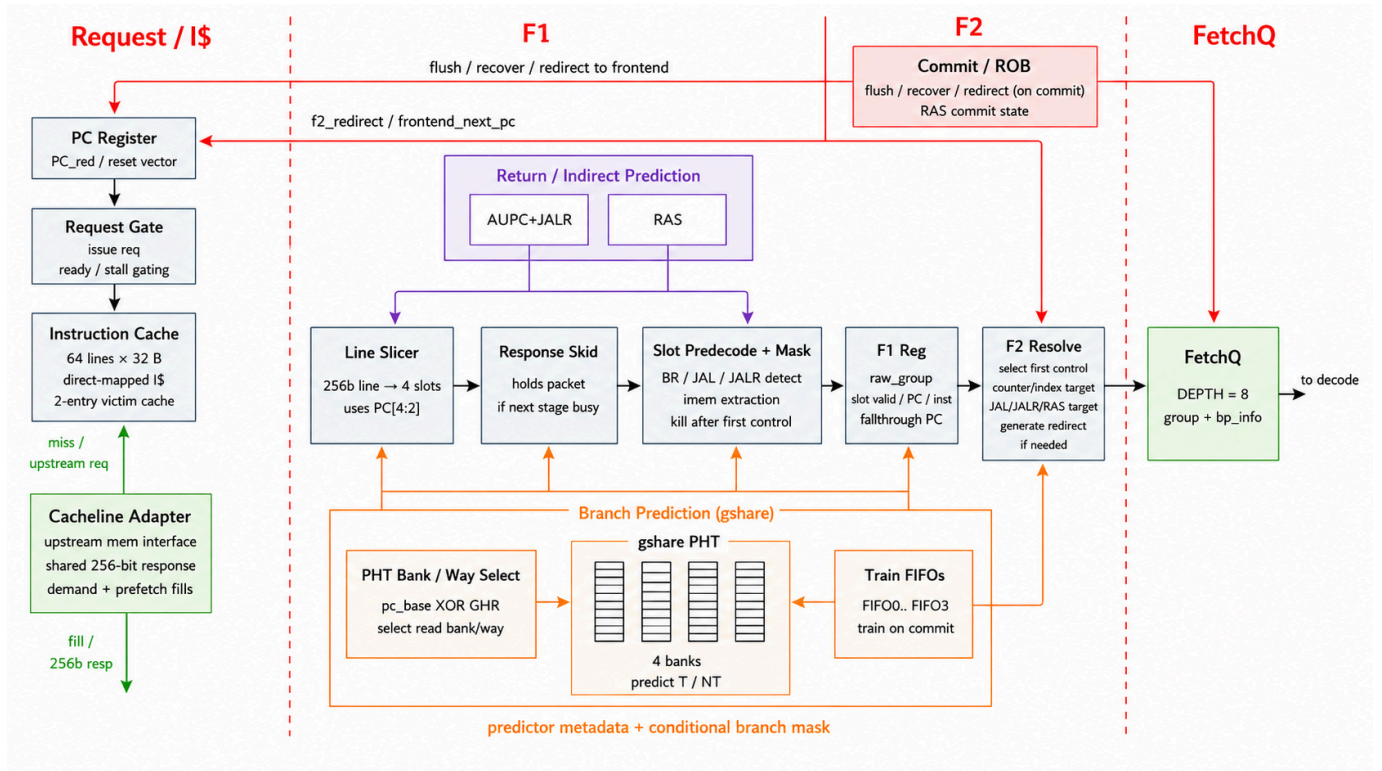


Fig. 3. Frontend fetch pipeline, predictor, redirect, and FetchQ organization.

return address stack pushes $pc + 4$ on calls and pops on returns. The RAS has speculative and committed head/count pointers, so flush recovery restores the speculative pointers from the committed state.

The second frontend stage forms the fetch queue entry and applies redirects. Resolving redirects here shortens the first-stage request path, but adds a late-redirect penalty for taken branches, jal, predicted jalr, and fall-through repair after masking younger slots.

The fetch queue is a 4-entry FIFO of 4-wide groups. Decode consumes the low half first. If the high half is valid, decode toggles *high_half* and consumes slots 2 and 3 on the next cycle. Branch metadata is remapped so a branch in slot 2 or 3 appears as backend slot 0 or 1 during high-half decode.

The I-cache is direct-mapped with 32-byte lines. A small victim cache stores recently evicted lines. On a victim hit, the line is swapped back into the I-cache and supplied to fetch.

The next-line prefetcher tracks line transitions. When fetch enters line N , it prefetches line $N+1$; when resources allow, it may also track $N+2$. Prefetches use separate MSHRs and check I-cache tags plus existing prefetch MSHRs to avoid duplicates. Returned next-line prefetches can install into the I-cache, while farther-ahead prefetches may stay buffered until demanded or promoted.

On a demand miss, the I-cache can consume a completed prefetch MSHR instead of issuing a new request. If the line is already in flight as a prefetch, demand fetch can take ownership of the response. This reduces duplicate memory traffic.

Overall, the frontend uses 4-wide fetch to refill the fetch queue quickly after bubbles, while the two-stage pipeline keeps the request path shorter. The main limitations are late redirects, no cross-line fetch packets, and only one predicted control-flow instruction per packet.

c) Performance Analysis:

The scalar frontend often left the fetch queue empty, so we first improved control-flow prediction.

Adding gshare improved CoreMark IPC from 0.32 to 0.39, a 1.22x speedup. This showed that conditional branch mispredictions were a major frontend bottleneck.

Adding the RAS plus *aupic/jalr* target prediction improved CoreMark IPC from 0.39 to 0.413, a 1.06x speedup over gshare alone. This reduced bubbles from returns and common *aupic/jalr* call sequences.

We also reduced predictor area. The original 1024-entry-per-bank, 1R1W predictor cost about 51k area. The final 256-entry-per-bank, 1RW predictor cost about 13k area. CoreMark branch accuracy only dropped from 93.1% to 92.9%, so this was a strong area/performance tradeoff.

The next bottleneck was fetch bandwidth. A single-stage 2-way fetch path became timing-critical because slicing, predecode, prediction, redirect logic, and next-PC generation were all in one cycle. Splitting fetch into two stages improved timing, but added a late-redirect penalty.

Expanding fetch from 2-way to 4-way improved CoreMark IPC from 0.74 to 0.78, a 1.05x speedup. The gain came from faster fetch queue refill after bubbles from I-cache misses, redirects, jal, jalr, and line-boundary packets.

The 4-wide frontend does not achieve four useful instructions every cycle. Branch-heavy packets lose slots because

younger instructions after the first control-flow instruction are masked. Line-boundary packets can also contain fewer than four valid instructions. Even with these limits, 4-wide fetch improved burst instruction supply.

The pipelined I-cache improved timing by separating request capture from response use. The next-line prefetcher reduced sequential I-cache miss penalties, and the victim cache reduced direct-mapped conflict misses.

Overall, the main frontend gains were:

- gshare: CoreMark IPC 0.32 → 0.39
- RAS + auipc/jalr: CoreMark IPC 0.39 → 0.413
- 2-way fetch → 4-way fetch: CoreMark IPC 0.74 → 0.78
- predictor area: 51k → 13k with only 93.1% → 92.9% CoreMark accuracy loss

The main remaining frontend limitations are late redirects, no cross-line fetch packets, one predicted control-flow instruction per packet, and remaining I-cache miss stalls.

D. 2-Way Superscalar Mid/Backend

a) *Motivation:* We implemented 2-way superscalar decode, rename, and dispatch stages. The main motivation for such a design was the observation that our instruction queues were often starved. Our performance trackers prior to a superscalar front and middle implementation showed instruction flow in bursts and with instruction queue occupancies in the 15% range. Around 60% of the time the ALU instruction queues were empty, while multiply and divide queues posted even worse occupancy rates. This caused us to conclude that there was a frontend throughput issue with our core. Upon making this data supported observation, we set out to improve this throughput.

b) *Design:* The following diagram describes our out of order superscalar middle and backend. We made every stage after fetch and up to dispatch 2 way superscalar in order to feed as many instructions to issue as possible.

While decode was a trivial change, register renaming logic required a significant amount of extra logic. We had to check 2 different types of hazards manually: WAR, WAW. Although RAW hazards are resolved by the issue queues and CDB dependence checking, we still had to check for WAR and WAW hazards. This was required as although explicit register renaming usually gets rid of the WAR and WAW hazards (technically not real hazards, just caused by out of order execution), this only applies when instructions pass through the rename/dispatch stages 1 per cycle. When two or more instructions try to use the structures in parallel, these dependencies must be checked manually to ensure correctness.

WAR: We manually checked whether the architectural destination register for slot 0 matched the architectural operand registers for slot 1. If they did we assigned the alias from the new PDST assigned for slot 0 rather than using the RAT to get operand physical registers.

WAW: Similarly, we manually checked whether slot 0 and slot 1 had the same architectural destination registers and assign the same physical destination registers to both.

After checking each of these hazards, we were able to make the rest of the rename/dispatch stages work except that we now required multiple ports to a plethora of structures including

the free list, busy table, RAT, RRAT (since we also doubled commit width), ROB, etc. We also had to double the pipeline registers between rename and dispatch as well as double the number of instructions enqueueable to the issue queues every cycle from 1 to 2.

In order to save area, we also converted the free list and busy list to bitvectors instead of queues/lists. In addition, we spent a day removing extra information from the ROB such that we could make it bigger without adding too much area.

At this point we were finished and happy with rename, dispatch and the structures directly connected.

Although the issue/PRF read stages stayed largely unchanged even after superscalar changes, we found many performance driven optimizations that could be made with our functional units in execute.

For one, we found that the new superscalar frontend and middle were able to feed many more instructions to the backend but in bursts. On some of such cycles, we noticed that the ALU issue queues filled up, stalling dispatch and therefore the rest of the processor frontend. To mitigate this, we added a second ALU and gave it its own issue queue and CDB. Although this increased area due to an extra PRF read port and extra ports in many structures due to the extra CDB, we deemed it useful enough to keep. We also found and implemented a multitude of divide and multiply optimizations thanks to our performance counters, the details of which are explained in the following section.

c) Performance Analysis:

Performance dictated many of the decisions we made with the backend.

The first had to do with our second ALU functional unit, which itself was driven by the observation that our ALU instruction queue was filling up in bursts. Specifically we noticed that about 45% of the time our issue queues were empty, but 9% of the time they were full. Although the easiest fix here was to increase the size of the instruction queues, this only reduced the full time from 9% to 8%. This told us that executing more ALU instructions in parallel would be a better solution.

This was our main motivation in adding the second ALU functional unit and instruction queue + CDB. Immediately, our IPC jumped by about 0.02 on coremark with almost no extra timing issues. Although our area increased, keeping both issue queues small but having two seemed to mitigate most of the extra area (it also helped that the ALU was a relatively small component in area).

In addition we noticed that the ALU utilization was quite unequal. Although alu0 saw usage rates around 60-70%, alu1 only saw usage rates around 30-40%. It appeared that instructions favored the first ALU rather than the second and the unequal utilizations was hurting performance for a few reasons.

One reason, we inferred, was that unequal utilization actively hurt us when one of the issue queues filled up, as if we only used the second ALU as overflow space we'd still have to wait for any RAW hazards from the instructions queued for the first ALU. This forced us to execute instructions roughly in

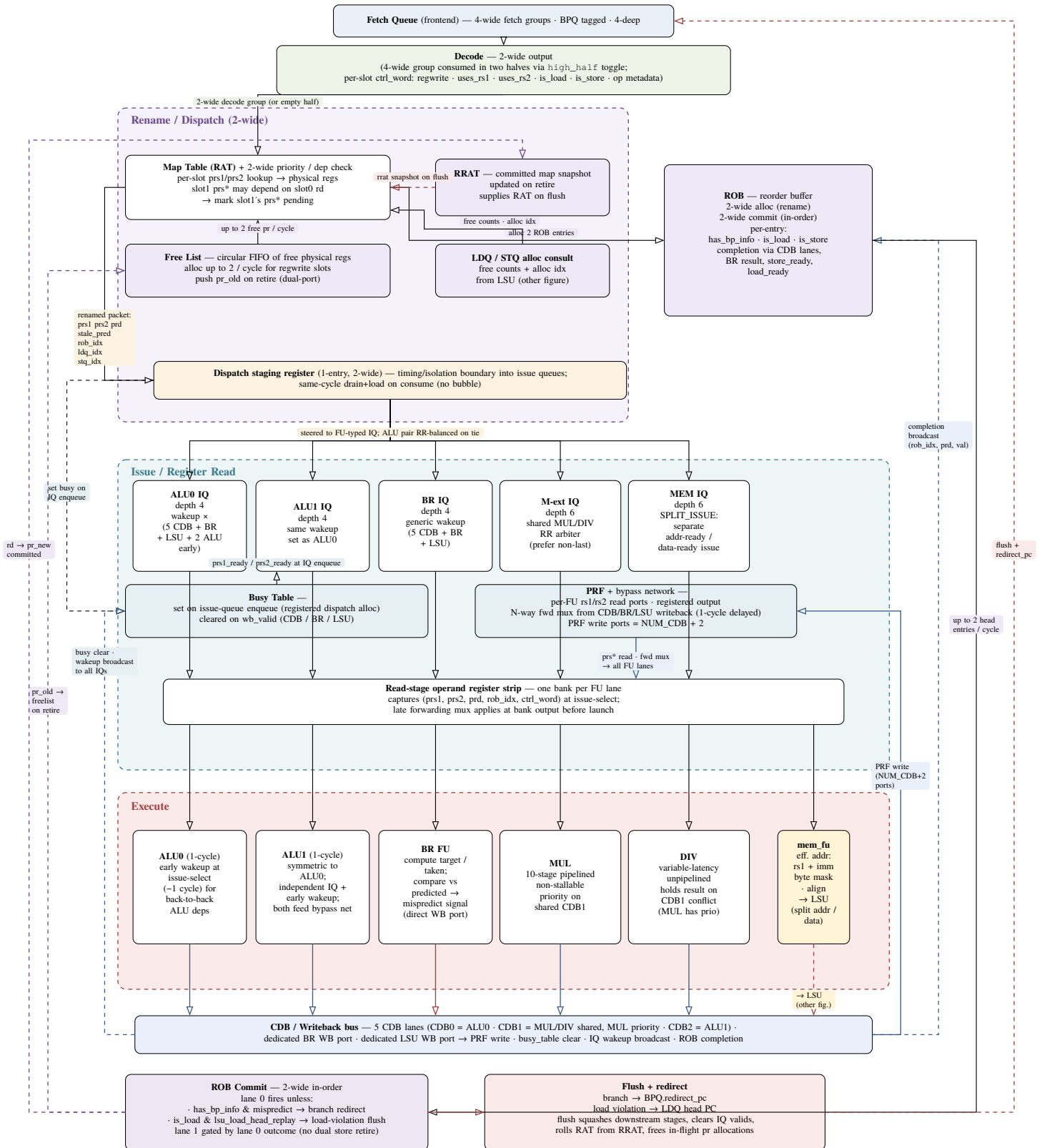


Fig. 4. Detailed backend microarchitecture from rename through commit (frontend abstracted, LSU/D\$ in companion figure). Solid black arrows are forward dataflow; navy solid arrows show PRF/bypass operands flowing through the read-stage strip into the FUs and PRF writeback from the CDB; red solid arrows are FU writebacks via the CDB; red dashed arrows are the BR/LSU writebacks (which bypass the CDB on dedicated WB ports) and the flush + redirect path; navy dashed arrows are the CDB back-loop carrying busy-table clears and IQ wakeup broadcasts, and the ROB completion broadcast; purple dashed arrows are commit-time updates to the architectural map (RRAT) and the free list. The Rename/Dispatch cluster fuses 2-wide priority-encoded RAT lookup, free-list allocation, LDQ/STQ index consult, and ROB allocation into a single dispatch packet, captured by a 1-entry isolation register before fanning out to FU-typed issue queues. The M-extension queue arbitrates a shared CDB lane between the pipelined MUL and the unpipelined DIV (MUL has priority on conflicts). Both ALUs broadcast wakeup at issue-select (one cycle ahead of writeback) so dependent ALU ops can issue back-to-back through the bypass network. BR and LSU each have dedicated WB ports, bypassing the shared CDB. Commit is 2-wide and in-order with lane-0 gating: a branch mispredict or load-violation replay on the head entry triggers flush + PC redirect, rolling RAT back from RRAT and freeing in-flight physical-register allocations.

TABLE II
AGGREGATE FINAL LEADERBOARD RESULT

fmax (MHz)	Area (μm^2)	Geomean PD ⁴
600.2	290124	66.354

TABLE III
PER-BENCHMARK FINAL PERFORMANCE RESULTS.

Benchmark	IPC	Delay (μs)	Power (mW)	PD ⁴
aes_sha	0.503	2568.4	56.178	2444.613
cnn	0.776	2898.5	57.163	4034.756
compression	1.548	462.7	59.565	2.730
coremark_im	0.810	599.9	57.642	7.464
fft	1.063	1819.1	57.570	630.377
graph	0.749	534.2	56.072	4.565
image	0.769	1025.2	56.050	61.923
mergesort	0.831	966.1	58.281	50.765
physics	0.684	1457.1	56.591	255.128
raytracing	0.825	1525.8	56.525	306.344
sudoku	0.554	340.2	57.688	0.773

order and removed many of the parallelization benefits from having two units.

Another reason might have been that even if queues weren't full, if we had multiple independent ALU instructions or multiple dependent instructions on a single instruction, our early wakeup logic could execute instructions much faster when there was a roughly equal split between the two ALUs.

Therefore to mitigate these problems we added a round robin to our dispatch stage such that each ALU instruction would alternate either being enqueued to alu0 or alu1. This changed our utilization to roughly 50%-50% and caused an IPC gain of around 0.02 on coremark once again.

In addition, we made multiple changes to the multiply and divide units. At first we had around 3 stages on our pipelined multiplier and a 5 cycle sequential divider. Although our IPC was really good (especially on benchmarks such as image or AES-SHA which were heavy on multiply and divide instructions) our timing was hurt quite a bit by having so few stages/cycles. To fix these critical paths we first had to disable many forwarding paths we had implemented for our multiplier and divider. Instead we had to replace these with stalls, losing quite a bit of IPC. We also converted the multiplier to 10 stages and the divider to a 16 cycle sequential divider. Timing was quite a bit better here, going from our critical path at under 500 MHz to rarely being a critical path problem again up until our peak frequency around 650 MHz.

In addition, we dispatched performance counters to log every bypass optimization we could possibly make in the multiplier and divider. Doing this allowed us to find optimizations such as multiply by 0, multiply by power of 2, multiply by 1, and other trivial or convertible operations for the multiply functional unit. These optimizations resulted in significant IPC changes for benchmarks like image, which went up by up to 0.03.

We made similar optimizations for the divider functional unit, including divide by 0, divide 0 as the dividend, divide by 1, dividing by power of 2, etc. Later, upon analysis of

benchmarks like image, we also decided to add a second divider which only operated on operands 16 bits x 16 bits. We did this as we had a performance counter which checked for the usage of upper bits. We noticed that even in the most division heavy benchmarks there were not many operations that used the upper 16 bits. Therefore adding a second divider was worth the area (which was only around 3k extra) for the IPC increase. We ensured it acted like just another bypass, keeping the same CDB and issue queue. Since all modifications remained inside the division functional unit, the extra area and timing paths were largely limited.

We also saw that multiply and divide instructions were quite sparse even in the heaviest benchmarks. Therefore to save CDB ports - and therefore area - we combined the multiply and divide functional units into a single CDB and implemented an arbiter to allow round robin instruction completion.

IV. FINAL EVALUATION RESULTS

EPYCfail finished with a final synthesized frequency of 600.2 MHz and an area of 290,124 μm^2 , staying under the 300k area limit while maintaining competitive performance across the benchmark suite. The final geomean PD⁴ was 66.354.

V. CONCLUSION

EPYCfail ultimately achieved its main design goal: implementing a functioning superscalar, out-of-order RV32IM processor with explicit register renaming, speculative control flow, and several memory-system and frontend optimizations. Beginning from a basic planned OoO design, we progressively built the rename, dispatch, issue, ROB, frontend, and LSU structures needed to support real instruction-level parallelism. The final core combined a 4-way fetch frontend, a 2-way renamed middle end, multiple issue queues, several functional units, 5 CDBs, a 2-wide commit ROB, branch prediction,

a return address stack, an instruction-cache prefetcher, and a speculative split LSQ with a non-blocking pipelined data cache.

The most important lesson from the project was that high-performance processor design is not only about adding more aggressive features, but about balancing those features against timing, area, and verification complexity. Some optimizations, such as the split LSQ, gshare predictor, return address stack, prefetcher, and second ALU, gave clear performance improvements. Others, such as the banked data cache, were implemented but ultimately removed because their performance benefit did not justify their timing or frequency cost. The split-LSQ required many timing optimizations as we pushed our frequency. Similarly, several structures were intentionally kept small, simplified, or pipelined because staying under the area limit and improving clock frequency became as important as improving IPC.

Our final design also showed how tightly coupled the frontend, backend, and memory system are in an out-of-order core. Early in the project, the backend was often starved by the frontend; later, after improving fetch bandwidth and superscalar dispatch, bottlenecks shifted toward memory ordering, D-cache contention, MSHR availability, ROB pressure, and critical paths in the LSU and fetch logic. This progression made performance counters especially valuable: they allowed us to distinguish features that were theoretically useful from features that addressed actual measured bottlenecks.

Overall, EPYCfail gave us a practical view into the trade-offs behind modern high-performance cores. Implementing explicit register renaming, speculative execution, branch prediction, superscalar fetch and dispatch, and a split LSQ required careful coordination across many structures that were individually complex and timing-sensitive. While the final processor still had clear future optimization opportunities, especially larger queues, more MSHRs, improved memory bandwidth, and a more flexible frontend, it successfully demonstrated a complete out-of-order superscalar design and the engineering compromises required to make such a design work within realistic area, timing, and verification constraints.